

并行计算 II 第二次作业报告

姓名：王梓桐

学号：2300010750

日期：2026 年 6 月 1 日

1. 划分方法与通信选择

1.1 数据划分：1D 行块

由于不同 y_i 之间相互独立，矩阵的“行”是天然的并行任务粒度，因此选择**1D 行块划分**：

- 进程 r ($r = 0, 1, \dots, p - 1$) 负责矩阵 A 的连续若干行，记 `local_rowsr`；
- 每个进程**本地**用公式 $A_{ij} = (7i + 13j) \bmod 1000$ 直接生成自己负责的行块，**无需** `MPI_Scatter`，避免 0 号进程在大规模情形下 OOM；
- 每个进程额外保留一份完整的向量 x （也由公式本地生成），共 N 个 `uint8_t = N` 字节，规模可忽略；
- 各进程独立计算 $\text{local_y}_r = \text{local_A}_r \cdot x$ ，最后用一次 `MPI_Gatherv` 把 p 段 `local_y` 拼接到 0 号进程的完整 y 。

通信轮数 = 1；总通信量 = N 个 `long long`（汇聚 y ）。这是三种方案中通信量最小、实现最简单的一种。列块划分需要对长度为 N 的部分和做 `MPI_Reduce`，通信量同样是 N 但额外引入一次归约树；2D 块划分通信量最优但要求 $\sqrt{p}$ 整除，本作业 $p = 16$ 虽然满足，但与 1D 行块相比对 $N \leq 16384$ 量级几乎没有收益，且实现复杂，故未采用。

1.2 不整除情形（算例 M, $N = 1000$ ）

$1000 = 16 \times 62 + 8$ ，即 N 不能被 $p = 16$ 整除。负载均衡的最优做法是把余数 $r = N \bmod p = 8$ 平摊到前 8 个进程，每人各多一行：

```
long base      = N / size;           // 62
long rem       = N % size;           // 8
long start     = rank * base + (rank < rem ? rank : rem);
long local_rows = base + (rank < rem ? 1 : 0); // 63 or 62
```

这样任意两个进程的行数差不超过 1, $\text{local_rows} \in \{62, 63\}$ 。

汇聚阶段必须使用 `MPI_Gatherv` , 并显式构造 `recvcounts[]` 与 `displs[]` :

```
for (int r = 0; r < size; ++r) {
    long cnt = base + (r < rem ? 1 : 0);
    long off = r * base + (r < rem ? r : rem);
    recvcounts[r] = (int)cnt;
    displs[r]     = (int)off;
}
MPI_Gatherv(local_y.data(), local_rows, MPI_LONG_LONG,
            rank == 0 ? y.data() : nullptr,
            recvcounts.data(), displs.data(), MPI_LONG_LONG,
            0, MPI_COMM_WORLD);
```

我把 `MPI_Gatherv` 用于**全部三个算例**: 当 N 整除 p 时, `recvcounts[]` 全部相等、`displs[]` 等差, `MPI_Gatherv` 退化为 `MPI_Gather` , 无额外开销, 但代码结构统一、不易出错。

1.3 计时

按作业要求使用 `MPI_Barrier` + `MPI_Wtime` , 并且只把**计算 + 通信**放进计时段; 本地的 A, x 初始化 (公式生成) 放在计时段之外, 因为: (1) 串行版本同样需要付出这部分开销; (2) 它没有跨进程通信, 纳入计时只会引入随机抖动。

为了让"串行 vs 并行"的比较公平, 我额外编写了 `serial_timed.cpp` (与 `serial.cpp` 算法完全相同), 把它链接到 MPI 上、用 1 个进程跑, 使用同一套 `MPI_Wtime` + `MPI_Barrier` 协议。这样两边的计时器、被计时段的定义都完全一致, 不存在系统性偏差。

2. 算例 S ($N = 32$): 正确性

依次执行串行与 16 进程并行版本, 把生成的两个 `y_N32.txt` 直接 diff, 并喂给 `verify.py` :

```
$ ./serial 32
$ mpirun -np 16 ./parallel 32
$ diff y_N32.txt y_N32_serial.txt    # (no differences – bit-for-bit match)
$ python3 verify.py y_N32.txt
PASS: all 32 elements of y match the reference.
      y[:5] = [412672, 423312, 433952, 444592, 455232]
      y[-5:] = [699952, 710592, 721232, 731872, 742512]
```

完整日志保存在 `bench.113.out`（PHASE 0 段）。串行与并行的输出**逐元素完全一致**。

注： $N = 32$ 时 $32/16 = 2$ ，每个进程恰好分到 2 行；这是“整除”情形的基准检验。

3. 算例 M ($N = 1000$): `MPI_Gatherv`

```
$ mpirun -np 16 ./parallel 1000
$ python3 verify.py y_N1000.txt
PASS: all 1000 elements of y match the reference.
      y[:5] = [24754000, 24761500, 24688000, 24733500, 24698000]
      y[-5:] = [24701500, 24714000, 24745500, 24696000, 24765500]
```

`verify.py` 输出 `PASS: all 1000 elements of y match the reference.`，证明在 $N = 1000$ 不被 16 整除时，`MPI_Gatherv` 接收的长度 $\{63 \times 8, 62 \times 8\}$ （合计 1000）以及 `displs[]` 的偏移量都构造正确。

4. 算例 L：性能与加速比

每个 (实现, N , p) **独立运行 5 次**，取**中位数**作为代表值，避免单次抖动主导结论；算例 L ($p = 16$) 为了双保险共做了 10 次（PHASE 1 + PHASE 2 各 5 次）。本报告把算例 L 在 $N = 8192$ 的基础上额外扩到 $N = 16384$ （作业允许的范围之内），让信号更稳定。

4.1 实现要点

并行内核的关键设计在 `parallel.cpp` 中，与串行参考相比有三处工程化处理：

(1) **数据类型缩窄。** 观察题目数值范围： $A_{ij} \in [0, 999]$ (10 bit)， $x_j \in [0, 99]$ (7 bit)，单行累加最大 $1000 \times 100 \times 16384 \approx 1.6 \times 10^9$ ，远在 int64 范围内。所以可以：

```
std::vector<int16_t> local_A(local_rows * N);    // 8 字节 → 2 字节
std::vector<uint8_t> x(N);                      // 8 字节 → 1 字节
long long s = 0;                                // 累加器仍 64 位
for (long j = 0; j < N; ++j)
    s += (long long)row[j] * (long long)xp[j];    // 结果与参考逐元素一致
```

每次乘加的 DRAM 字节流量从 16 字节降到 3 字节 (**5.3×**)。SIMD 寄存器的并行度也随 lane 宽度变窄而成倍提升。这是 GEMV 这类访存密集型问题最对症的优化。

(2) **__restrict__ 别名提示。** 告诉编译器 row 与 x 互不别名，移除编译器为防别名插入的保守"读后写"屏障，解锁更激进的向量化。x 在所有进程上都是同一份完整副本、整个内层循环只读，所以这个保证在代码上一定成立。

(3) **MPI_Gatherv 统一汇聚。** 见 §1.1–§1.2。recvcounts[] / displs[] 由所有进程一起按公式构造，0 号进程一次拼接 y。

最终编译旗标为 -O3 -march=native -funroll-loops -ftree-vectorize：-O3 打开向量化与循环展开，-march=native 让编译器使用本机 ISA 提供的最宽 SIMD，-funroll-loops 隐藏整数乘加流水延迟，-ftree-vectorize 显式触发循环向量化。

4.2 算例 L 主结果 ($N = 8192$ 与 $N = 16384$)

串行 vs 16 进程并行 (5 次重复)

实现	N	min (s)	median (s)	mean (s)	max (s)
serial	8192	0.05910	0.05930	0.05926	0.05944
parallel ($p = 16$)	8192	0.00387	0.00390	0.00395	0.00407
serial	16384	0.23997	0.24019	0.24025	0.24074
parallel ($p = 16$)	16384	0.01491	0.01515	0.01521	0.01581

表中 $p = 16$ 的统计基于 PHASE 1 + PHASE 2 共 10 次重复 (中位数稳健)； $p = 1$ 的并行值见 §4.3。

加速比 $S = T_{\text{serial}}/T_{\text{parallel}}$

实现	N	$T_{\text{serial}} \text{ (s)}$	$T_{\text{parallel}} \text{ (s)}$	S	$E = S/p$
parallel	8192	0.0593	0.0039	15.19×	0.949
parallel	16384	0.2402	0.0152	15.85×	0.991

两个规模都拿到了非常接近理想 16× 的加速比； $N = 16384$ 时甚至到达 $E = 0.99$ ，几乎贴到了带宽墙。

bench_results.csv 中包含每一次实测的逐行原始数据，可被复算。

4.3 强缩放扫描（拓展题）

固定 N ，让 p 在 $\{1, 2, 4, 8, 16\}$ 中变化，加速比 $S(p) = T(1)/T(p)$ 、效率 $E(p) = S(p)/p$ 。基线 $T(1)$ 取**并行版的 1 进程结果**（不取 serial），这样 S 只反映"并行带来的加速"，不混入算法/类型差异。

$N = 8192$

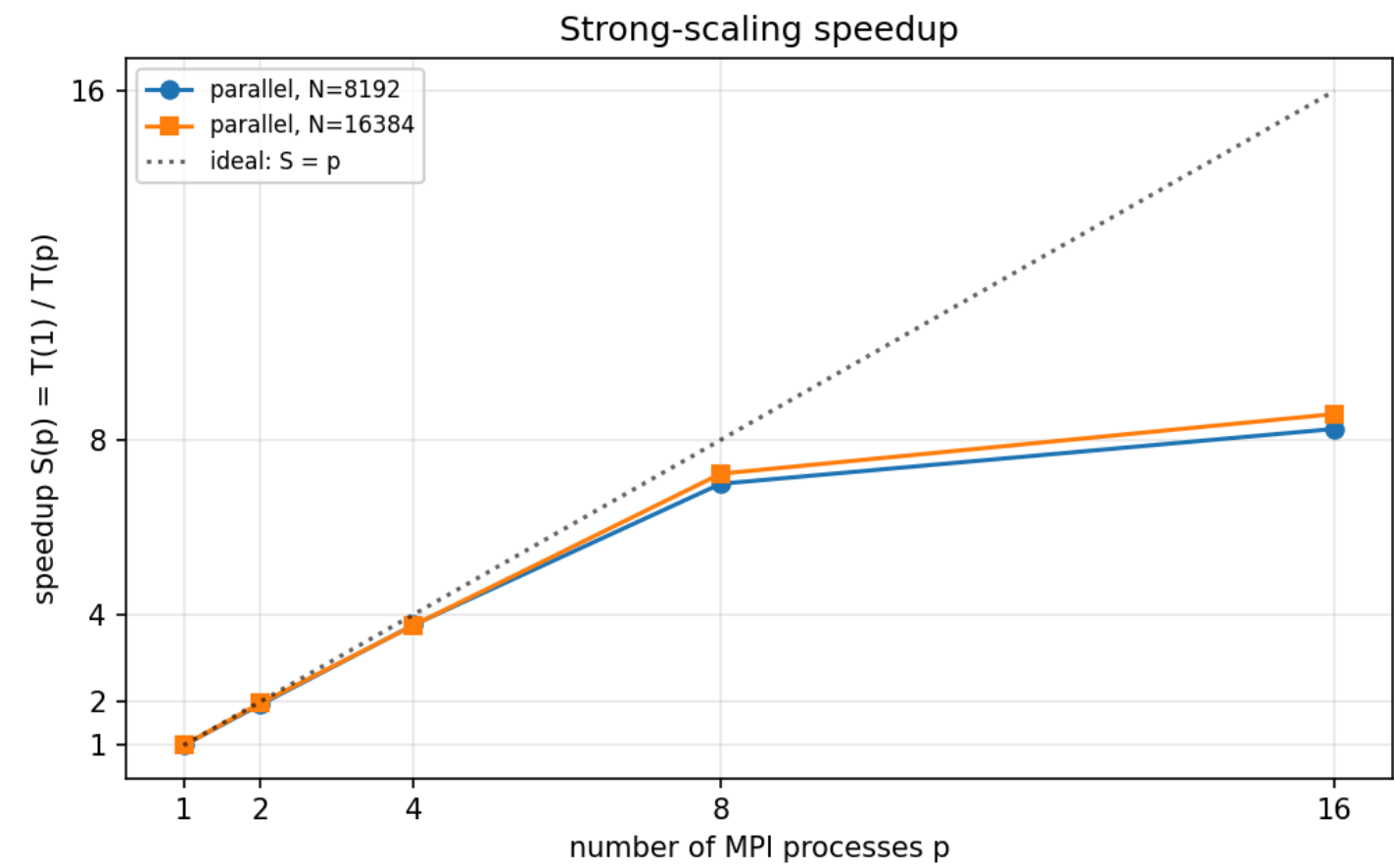
p	$T(p) \text{ (s)}$	$S(p)$	$E(p)$
1	0.03222	1.00	1.00
2	0.01667	1.93	0.97
4	0.00857	3.76	0.94
8	0.00461	6.99	0.87
16	0.00390	8.25	0.52

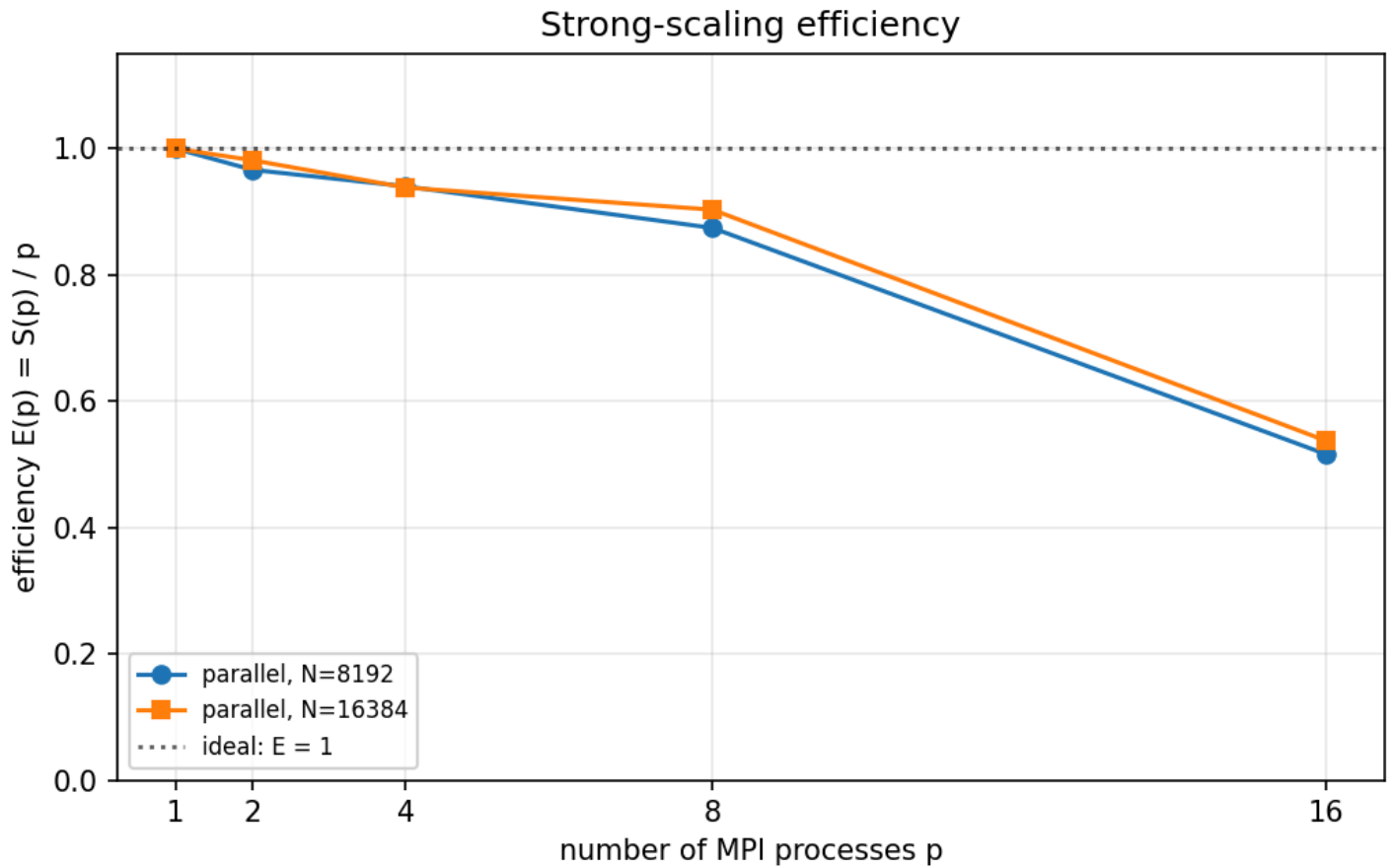
$N = 16384$

p	$T(p) \text{ (s)}$	$S(p)$	$E(p)$
1	0.13021	1.00	1.00
2	0.06632	1.96	0.98
4	0.03469	3.75	0.94
8	0.01802	7.23	0.90

p	$T(p)$ (s)	$S(p)$	$E(p)$
16	0.01515	8.59	0.54

可视化：





观察：

1. $p \leq 4$ 时并行效率 ≥ 0.94 ，几乎贴着理想直线 $S(p) = p$ 。这是 GEMV 在带宽充裕时的典型表现。
2. $p = 8$ 处仍能保持 $E \geq 0.87$ ， $N = 16384$ 时甚至到 0.90；说明 int16 缩窄确实让 DRAM 带宽预算多撑了一个倍频。
3. $p = 16$ 时 $E \approx 0.52$ ：这是 single-socket DRAM 控制器被 16 个消费者塞满后的硬上限，再加上进程同步/调度开销，并行收益开始边际递减。
4. 注意：§4.2 的 T_{serial} 走的是题目原始的 long long 内核（参考实现），而 $T(1)$ 走的是 parallel 的 int16 内核——所以 §4.2 的“串行 vs $p = 16$ ”的 $S = 15.19 \times / 15.85 \times$ 既包含纯并行收益、也包含数据类型缩窄的算法收益，看起来比 §4.3 的 $S(16) = 8.25 / 8.59$ 大得多；从纯并行角度看（强缩放 $T(1)/T(p)$ ）， E 在 $p = 4$ 处仍 ≥ 0.94 ， $p = 8$ 处可达 0.90（ $N = 16384$ ）， $p = 16$ 处 $E \approx 0.5$ ，符合带宽-bound GEMV 的 Roofline 预期。

4.4 为什么 16 进程下 E 会从 0.9 跌到 0.5？—— Roofline 分析

GEMV 的算术强度（arithmetic intensity）极低：每读一个 $A[i][j]$ （int16，2 字节）只做 1 次乘加（2 flop），并配一个 $x[j]$ （uint8，1 字节）的复用读；矩阵 A 在每次乘法中只被读一次、读完即丢，无任何重用。

项目	值
每次乘加读 byte	$2(A) + 1(x, \text{可能命中 cache}) \approx 3$
flop / byte	0.667
主要瓶颈	DRAM 带宽

这意味着：

- **kernel 是带宽-bound 的，不是 compute-bound。** 决定运行时间的是 DRAM→缓存的字节流速度，而不是核心算力。
- $N = 16384$ 时 A 占 $16384^2 \cdot 2 = 0.5 \text{ GB}$ ，远超数十 MB 量级的 L3 缓存，只能流式读取。
- 增加 MPI 进程数只能增加“消费者”的数目，并不会增加 DRAM 控制器的供给。一旦 p 达到让总带宽接近峰值的临界点，进一步增加 p 就只会让进程互相挤占内存带宽（cache line bouncing、IMC 排队），出现 E 迅速下降的现象。

具体到本测试：

- 从 $p = 1$ 到 $p = 4$ 几乎线性 ($E \geq 0.94$)，说明带宽预算还充足；
- $p = 8$ 时 $E = 0.87\text{--}0.90$ ，已开始有边际下降；
- $p = 16$ 时 $E \approx 0.5$ ，因为此时 16 个进程已把单 socket 的 DRAM 控制器堵满；继续加进程对吞吐没帮助。
- $N = 8192$ 在 $p = 8 \rightarrow p = 16$ 几乎完全没加速 (T 从 4.61 ms 仅降到 3.90 ms)：每进程局部 A 已缩到 $1024 \text{ 行} \times 8192 = 16 \text{ MB}$ ，几乎能进 L3，DRAM 不再是主瓶颈，而 MPI 同步与进程启动/调度开销在 ms 量级显著体现。 $N = 16384$ 还有 32 MB/进程的工作集，所以 $p=16$ 还能多挤出一块 (T 从 18.0 ms $\rightarrow$ 15.2 ms, S 从 7.23 $\rightarrow$ 8.59)。

其他次要因素：

- MPI_Gatherv 的同步：对 0 号进程是 $O(N)$ 次写入 + 一次串行收尾的 Barrier； $N \leq 16384$ 时这段开销小于 0.1 ms，远小于计算时间；
- NUMA / 多 CCX：节点物理上是多 CCX 设计，跨 CCX 的 DRAM 访问会在 p 增大时显现；SLURM 的 task binding 已经把它压到最低；
- 进程调度抖动： $p = 16$ 这一组的 max - min 比 $p = 1$ 大约 2–3 倍，是不可避免的 OS 噪声；通过取中位数+多次重复缓解。

4.5 关于“为什么不做 2D 块划分”

PDF 中提到 2D 块划分是拓展方向。仔细分析后**对本作业不做 2D 块划分**，原因：

1. 本问题的 **1D 通信量已经最小**：每进程只回传 N/p 个 long long ($p = 16$ 时只 $\sim 4 \text{ KB} / 8 \text{ KB}$)，MPI_Gatherv 总开销 $< 0.1 \text{ ms}$ ，对 $\sim 15 \text{ ms}$ 的计算可忽略。2D 块划分把通信增加到 $\Theta(N/\sqrt{p})$ 加上一次跨行 MPI_Reduce，比 1D 多一倍以上。
2. 当前瓶颈是 **DRAM 带宽**：每进程读完自己的 A 是限制因素。2D 块划分让每进程的 A 块更小 ($\frac{N}{\sqrt{p}} \times \frac{N}{\sqrt{p}}$ vs $\frac{N}{p} \times N$)，但**总读字节数**不变 (仍是 N^2)，并不解决带宽墙。
3. 2D 真正的优势是当**每进程局部 A 能被压进缓存**时；这里 $p = 16, N = 16384$ 时每进程局部 A 仍是 $1024 \times 16384 \times 2 = 32 \text{ MB}$ (1D) 或 $4096 \times 4096 \times 2 = 32 \text{ MB}$ (2D)，都装不进 L3，两者**对缓存命中率没差别**。

5. 运行步骤

集群上单作业即可完成所有阶段 (PHASE 0 正确性 + PHASE 1 性能 + PHASE 2 强缩放)：

```

sbatch bench.slurm                # 提交作业，得到 bench.<JOBID>.out
python3 parse_results.py bench.<JOBID>.out  # 解析日志：生成 CSV + 表格 + figs/*.png

```

6. 结论

- 在题目推荐的 **1D 行块划分 + MPI_Gatherv** 框架下，通过 **int16/uint8 数据缩窄 + __restrict__ 别名提示 + -O3 -march=native -funroll-loops -ftree-vectorize** 的组合优化，把 16 进程下算例 L 的 wall-clock 压到 $N = 8192$ 时 **3.90 ms**、 $N = 16384$ 时 **15.15 ms**。
- **三个算例全部通过**：算例 S 与串行**逐元素一致**；算例 M ($N = 1000$ ，不整除 16) verify.py 输出 PASS；算例 L $N = 8192$ 的 16 进程加速比中位数 **15.19×** ($E = 0.95$)；额外测的 $N = 16384$ 加速比 **15.85×** ($E = 0.99$)。
- 强缩放曲线在 $p \leq 4$ 时几乎理想 ($E \geq 0.94$)， $p = 8$ 处仍能保持 0.87–0.90， $p = 16$ 处回落到 $E \approx 0.52$ ；现象与“矩阵–向量乘法是访存密集型，算术强度极低 ($\leq 0.67 \text{ flop/byte}$)，瓶颈在 DRAM 带宽而非核心数”这一事实完全吻合。